CLOUD-06: GCP Integration

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 6 of 8 | **Created:** March 2026 | **Last Updated:** 08/12/2026

Overview

This notebook covers Dynatrace integration with Google Cloud Platform (GCP). You will learn how to configure GCP monitoring, authenticate via service accounts, explore supported GCP services, query GCP entities and metrics, and monitor GKE and Cloud Run workloads.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	`metrics.read`, `entities.read`, `logs.read`
GCP Project	With service account configured for Dynatrace
Connection	Clouds app or Helm-based GKE integration (recommended for SaaS) or Environment ActiveGate (classic)
Prior Knowledge	CLOUD-01 fundamentals

1. GCP Integration Architecture

Dynatrace integrates with GCP through the Cloud Monitoring API (formerly Stackdriver). For SaaS deployments, the recommended approach uses a **push-based Pub/Sub integration** deployed via Helm on GKE — no ActiveGate required.

Connection Methods

Method	Mechanism	Best For
Helm on GKE (push-based)	Metrics and logs forwarded via Pub/Sub to Dynatrace API	SaaS deployments (recommended)
Classic ActiveGate polling	Queries Cloud Monitoring API	Managed deployments, non-GKE environments
Cloud Functions + Pub/Sub	Forwards logs via Pub/Sub trigger	Log-only ingestion

Data Flow (Modern — Push-Based)

```
GCP Resources → Cloud Monitoring → Pub/Sub → Dynatrace Integration (on GKE) → Dynatrace SaaS
GCP Resources → Cloud Logging    → Pub/Sub → Dynatrace Integration (on GKE) → Dynatrace SaaS
```

Helm Deployment Options

Option	Cluster Type	Notes
New GKE Autopilot cluster	Autopilot	Recommended — Dynatrace manages the cluster
Existing GKE cluster	Standard or Autopilot	Deploy into an existing cluster

The Helm deployment creates the necessary Pub/Sub subscriptions, IAM roles, and forwarding components automatically.

GCP-Specific Considerations

- **Project-based organization** — GCP organizes resources by project, unlike AWS regions or Azure subscriptions
- **Labels vs tags** — GCP uses "labels" (equivalent to AWS tags / Azure tags)
- **Cloud Monitoring quotas** — API call limits apply per project
- **Pub/Sub costs** — Message volume determines Pub/Sub costs; filter at source to reduce volume

Forwarder Sizing and Throughput

For a high-volume GCP estate, size the Pub/Sub log forwarder **before** turning on high-cardinality sources — unfiltered volume above the forwarder's ceiling backs up silently in the Pub/Sub subscription.

Deployment	Throughput	Notes
Base resources	~8 GB/hour	Beyond this, messages are retained in the Pub/Sub subscription

Deployment	Throughput	Notes
Vertical — 4 vCPU / 4 Gi, 1 replica	~8 MB/s (~480 MB/min)	Dynatrace does **not** recommend scaling vertically — env vars must be re-tuned per machine
Horizontal — 4 replicas (4 vCPU / 4 Gi)	~25 MB/s (~2 TB/day)	Preferred scaling axis
Horizontal — 6 replicas (4 vCPU / 4 Gi)	~46 MB/s (~4 TB/day)	
Autoscaling	Recommended only above ~450 MB/min on a 4 vCPU / 4 Gi machine	

Sizing method: estimate your post-filter TB/day, size the replica count to match, and filter at the **Log Sink** (source) plus **OpenPipeline** to control back-pressure and cost. Filtering at the source is the cheapest lever — it lowers both Pub/Sub cost and forwarder load.

> **Source:** Set up GCP log integration — logs only (DT docs)

2. Service Account Authentication

GCP integration requires a service account with appropriate IAM roles.

Setup Steps

1. **Create a service account** in the GCP project
2. **Assign IAM roles** (see table below)
3. **Generate a JSON key** for the service account
4. **Upload the key** to Dynatrace Settings > Cloud and virtualization > GCP

Required IAM Roles

Role	Purpose
`roles/monitoring.viewer`	Read Cloud Monitoring metrics
`roles/compute.viewer`	Read Compute Engine resources
`roles/container.viewer`	Read GKE clusters and workloads
`roles/cloudasset.viewer`	Read asset inventory for entity discovery

Multi-Project Monitoring

To monitor multiple GCP projects with a single integration:

Approach	How It Works
Service account per project	Each project has its own SA; configure multiple integrations in Dynatrace
Cross-project service account	Grant SA roles at the organization/folder level; single integration
Monitoring scope	Configure a Cloud Monitoring metrics scope to aggregate across projects

> **Best Practice:** Use a dedicated GCP project for monitoring resources (service accounts, Pub/Sub topics) to keep billing and IAM clean.

3. Supported GCP Services

Category	Services
Compute	Compute Engine (GCE), Cloud Functions, App Engine
Containers	GKE (Google Kubernetes Engine), Cloud Run
Database	Cloud SQL, Cloud Spanner, Bigtable, Firestore
Storage	Cloud Storage, Persistent Disk

Category	Services
Data	BigQuery, Pub/Sub, Dataflow, Dataproc
Networking	Cloud Load Balancing, Cloud CDN, Cloud DNS
AI/ML	Vertex AI, Cloud TPU

Service Metric Availability

Service	Key Metrics	Typical Delay
Compute Engine	CPU, memory, disk, network	3-5 minutes
GKE	Container CPU/memory, pod status	1-3 minutes
Cloud Run	Request count, latency, instance count	1-3 minutes
Cloud Functions	Execution count, duration, memory usage	1-3 minutes
Cloud SQL	CPU, connections, storage, replication lag	3-5 minutes
BigQuery	Slot utilization, query count, bytes processed	5-10 minutes

4. Querying GCP Entities

List GCP-Hosted Entities

GCP compute instances that run OneAgent appear as regular host entities. Cloud-specific entities appear under their dedicated entity types.

```
// List all hosts (includes GCE instances with OneAgent)
fetch dt.entity.host
| fieldsKeep id, entity.name, tags
| sort entity.name asc
| limit 20

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | fieldsKeep id, name, tags
//   | sort name asc
//   | limit 20
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via
getNodeField).
```

List Kubernetes Clusters (Including GKE)

```
// List all Kubernetes clusters (GKE clusters appear here)
fetch dt.entity.kubernetes_cluster
| fieldsKeep id, entity.name, tags
| sort entity.name asc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fieldsKeep id, name, tags
//   | sort name asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via
getNodeField).
```

GCP Service Entity Count

```
// Count cloud application entities (includes GKE workloads, Cloud Run)
fetch dt.entity.cloud_application
| summarize resource_count = count()
| fieldsAdd resource_type = "Cloud Applications (GKE/Cloud Run)"

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_DEPLOYMENT"
//   | summarize resource_count = count()
//   | fieldsAdd resource_type = "Cloud Applications (GKE/Cloud Run)"
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
```

5. GKE Monitoring

Google Kubernetes Engine (GKE) is monitored with the DynaKube Operator, similar to EKS and AKS.

GKE-Specific Considerations

Feature	Monitoring Impact
Autopilot mode	No node-level access; use ApplicationMonitoring mode
Standard mode	Full node access; use CloudNativeFullStack mode
Workload Identity	Recommended for GKE pod IAM (replaces service account keys)
GKE Dataplane V2	eBPF-based networking; compatible with Dynatrace

GKE Container Metrics

```
// Top 10 containers by CPU usage in the last hour
timeseries containerCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h,
by:{k8s.container.name}
| fieldsAdd avgCpu = arrayAvg(containerCpu)
| sort avgCpu desc
| limit 10
```

GKE Node Utilization

```
// Node CPU utilization across GKE nodes over the last hour, derived at container grain.
//
// `dt.kubernetes.node.cpu_usage` and `dt.kubernetes.node.memory_working_set` DO NOT EXIST
// (verified 08/12/2026 against a full 836-key catalog enumeration). The `dt.kubernetes.node.*`
// namespace is real, but it publishes CAPACITY and CONDITION metrics — `cpu_allocatable`,
// `memory_allocatable`, `pods_allocatable`, `conditions` — not utilization. Node-level
// utilization is derived by summing the container metrics per node, which is what this cell does.
// A timeseries against a missing key returns an EMPTY result rather than an error, so the old
// cell simply drew nothing. Enumerate the real catalog with:
//   metrics | filter startsWith(metric.key, "dt.kubernetes") | summarize n = count(), by:{metric.key} | sort metric.key asc
// (`metrics` takes `from:` with NO leading comma; `summarize` requires an aggregation, and a
//  bare `metrics | fields metric.key` is capped and will silently under-report the catalog.)
timeseries used = sum(dt.kubernetes.container.cpu_usage), from:-1h, by:{dt.entity.kubernetes_node}
| fieldsAdd avgCpu = arrayAvg(used)
| sort avgCpu desc
| limit 10
```

GKE Pod Events

```
// Kubernetes events by reason over the last 6 hours.
//
// Corrected 08/11/2026 — this cell previously carried three separate filters
that each
// matched nothing, so it returned zero rows against an estate emitting
thousands:
//   1. `event.kind == "K8S_EVENT"` — event.kind never takes that value.
Verified over 24h
//      the only values are DAVIS_EVENT, SYNTHETIC_EVENT, FLEET_EVENT and
DAVIS_PROBLEM.
//      Kubernetes events arrive as DAVIS_EVENT; the discriminator is
event.provider.
//   2. `event.type == "Warning"` — every KUBERNETES_EVENT record carries
CUSTOM_INFO
//      (3,571 of 3,571 checked). Severity lives in
dt.kubernetes.event.important.
//   3. `event.reason` — null on every record. The real field is
dt.kubernetes.event.reason.
// Each filter was valid syntax that executed cleanly, which is why this
survived review.
fetch events, from:-6h
| filter event.provider == "KUBERNETES_EVENT"
| filter dt.kubernetes.event.important == "true"
| summarize event_count = count(), by:{dt.kubernetes.event.reason}
| sort event_count desc
| limit 10
```

6. Cloud Run Analysis

Cloud Run is GCP's serverless container platform. It shares some monitoring patterns with Lambda but runs containers instead of functions.

Cloud Run Monitoring Points

Metric	Description	Alert Threshold
Request count	Total HTTP requests	Baseline deviation
Request latency	Response time (p50, p95, p99)	SLO-dependent

Metric	Description	Alert Threshold
Instance count	Active container instances	Max instance limit
CPU utilization	Per-instance CPU usage	> 80% sustained
Memory utilization	Per-instance memory	> 80% (risk of OOM)
Startup latency	Cold start time for new instances	Application-dependent

Cloud Run vs Lambda

Aspect	Cloud Run	AWS Lambda
Unit	Container	Function
Runtime	Any Docker image	Managed runtimes + custom
Max duration	60 minutes	15 minutes
Concurrency	Multiple requests per instance	One per execution (default)
Monitoring	OneAgent (Java/Node only) or OTLP	Lambda Layer
Cold start	Container pull + startup	Runtime initialization

Cloud Run Monitoring: OneAgent Coverage and Caveats

OneAgent monitoring of Cloud Run managed is **limited to Java and Node.js**. Cloud Run itself runs any container image, but OneAgent auto-injection does not cover every runtime — other runtimes (Go, Python, .NET) send telemetry via **OTLP** instead.

Aspect	Detail
Supported runtimes (OneAgent)	Java and Node.js only — other runtimes use OTLP-direct
Credentials	Baked in as **build-time image arguments** (`DT_API_URL` , `DT_API_TOKEN`) — repointing to a new tenant requires an image rebuild and a new revision
Execution environments	Gen1 vs Gen2 — **Gen1 does not expose CPU/memory metrics** (intentional security limits); Gen2 has no such restriction

Aspect	Detail
Where instances appear	On the **Hosts** page (with GCP properties and each instance's memory limit), not the Container groups page
Cold-start overhead	Agent injection adds startup overhead; because each revision autoscales from zero, cold starts appear more often than on always-on hosts

> **Migration note:** because Cloud Run credentials are build-time image args, redirecting a Cloud Run workload to a new Dynatrace tenant means rebuilding the image and deploying a new revision — see **M2S-05: Step 5 — Execute** for the full serverless/container redirect matrix.
>
> **Source:** Monitor Google Cloud Run managed (DT docs)

```
// Cloud Run service spans in the last hour
//
// Corrected 08/12/2026: `cloud.platform` is DEPRECATED in the semantic
dictionary and carried no
// value on any span in the validation tenant, so `cloud.platform ==
"gcp_cloud_run"` could only
// ever return nothing. `cloud.provider` is the stable replacement (aws /
azure / gcp / ...).
// Note this narrows to provider, not service — add a service.name or
faas.name filter to isolate
// Cloud Run specifically. On a tenant with no GCP workloads this correctly
returns no rows.
fetch spans, from:-1h
| filter span.kind == "server" and cloud.provider == "gcp"
| summarize {avg_duration_ms = avg(duration) / 1ms, request_count = count()},
by:{service.name}
| sort request_count desc
| limit 10
```

Tracing Across Pub/Sub

GCP shops frequently place Pub/Sub between services. **Pub/Sub is absent from OpenTelemetry's auto-instrumentation coverage** where Kafka and RabbitMQ are first-class (the OpenTelemetry Java supported-libraries list carries Kafka `0.11+` and RabbitMQ `2.7+`, but no Pub/Sub entry) — so trace context is not propagated for you, and

a distributed trace breaks silently at every Pub/Sub hop unless you propagate it yourself. Coverage varies by language and client version; verify for your own runtime.

Carry the W3C `traceparent` through the Pub/Sub message `attributes` map:

- **Publisher:** inject the active trace context into `message.attributes` (a `traceparent` attribute) before publishing.
- **Subscriber:** read `traceparent` from `message.attributes` and use it as the parent when starting the consumer span.

Without this, producer and consumer spans appear as disconnected traces. See the **SPANS** series for span and trace-context fundamentals.

> **Sources:** Span and trace context propagation (DT docs); OpenTelemetry Java supported libraries (OpenTelemetry GitHub)

7. GCP-Specific Entity Mapping

GCP uses a project-based hierarchy that maps to Dynatrace as follows:

GCP Construct	Dynatrace Mapping	Notes
Organization	Account-level (manual)	Not auto-discovered
Folder	Tag or Management Zone	Organizational grouping
Project	Tag (`gcp.project`)	Primary scoping unit
Region / Zone	Tag or entity property	Geographic placement
Labels	Dynatrace tags	Key-value metadata

Best Practices for GCP Mapping

Practice	Description
Tag by project	Ensure GCP project ID is propagated as a Dynatrace tag
Use label conventions	Standard labels: `env` , `team` , `service` , `cost-center`
Segment by project	Create Dynatrace segments per GCP project for access control

Practice	Description
Monitor billing exports	Forward BigQuery billing data to Dynatrace for cost correlation

8. Summary and Next Steps

Key Takeaways

- GCP integration authenticates via **service account JSON keys** with Monitoring Viewer and Compute Viewer roles
- **GKE Autopilot** requires ApplicationMonitoring mode; **GKE Standard** supports CloudNativeFullStack
- **Cloud Run** is monitored with OneAgent (Java/Node only; other runtimes via OTLP) injected at image build time; instances appear on the **Hosts** page
- GCP **projects and labels** should map to Dynatrace tags and segments for consistent governance

Next Steps

- **CLOUD-07: CloudWatch Log Ingestion** — Log forwarding patterns (applicable to GCP via Pub/Sub)
- **CLOUD-08: Multi-Cloud Patterns** — Unified monitoring across GCP and other providers

---